# Déploiement d’un nœud Ethereum

### Rapport de l’atelier · rendu du 31 octobre 2026

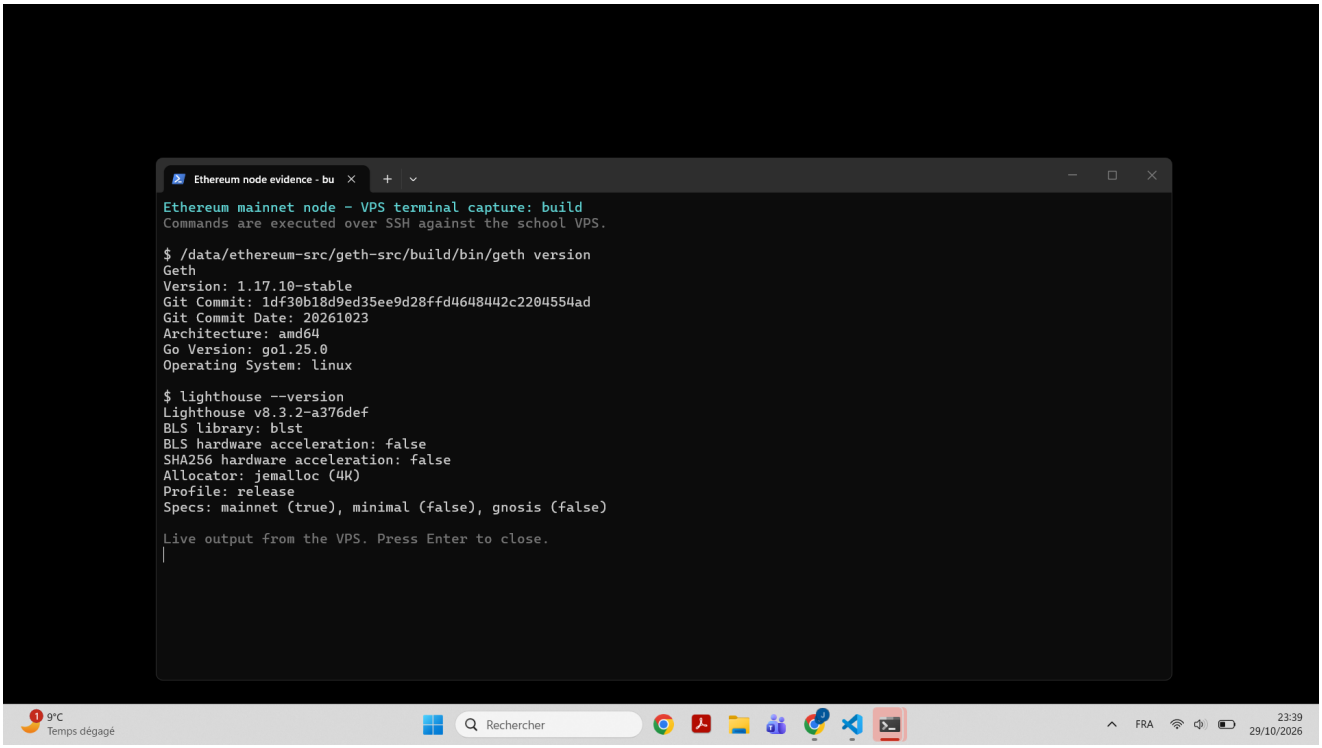
| | |
|---|---|
| **Étudiant** | Jordi PARIS |
| **VM** | TP-MESIFI590426-07 |
| **Réseau** | Ethereum mainnet |
| **Clients** | Geth v1.17.10 et Lighthouse v8.3.2 |
| **Captures prises** | 29/10/2026 23:41 |

## 1. Objectif et choix des clients

Pour ce travail, j’ai déployé un nœud Ethereum composé d’un client d’exécution et d’un client de consensus. J’ai choisi Geth pour l’exécution, qui prend en charge le mode snap, et Lighthouse pour le consensus, qui fournit le client Beacon et la synchronisation par point de contrôle. Les deux logiciels ont été compilés sur le VPS depuis leurs sources, sans Docker ni binaire client précompilé.

## 2. Préparation et compilation

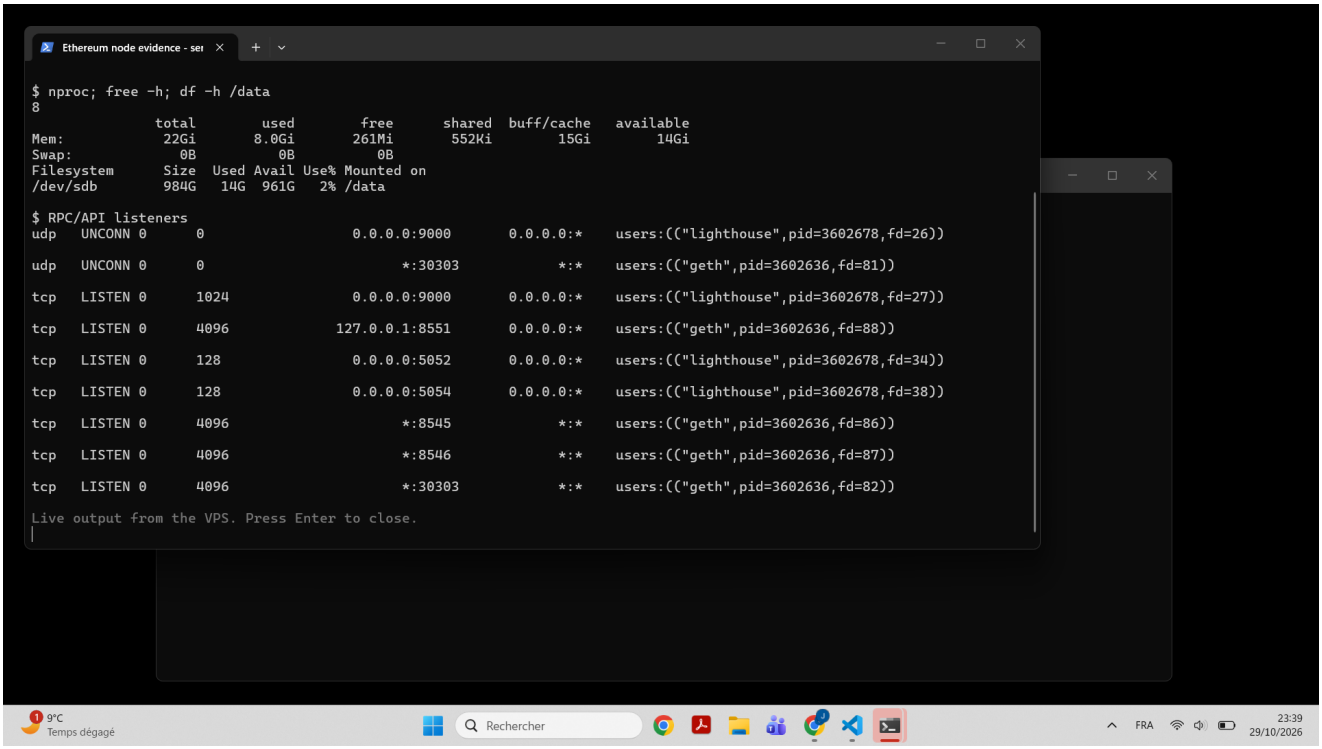
La VM utilise Debian 13, 8 vCPU et 22 Gio de mémoire. J’ai installé Git, Go, Rust, les compilateurs C/C++, Clang, CMake, Protobuf et les bibliothèques nécessaires. Rust fourni par Debian était trop ancien pour Lighthouse v8.3.2 ; le toolchain stable a donc été installé avec rustup. Geth v1.17.10 a été construit avec make geth et Lighthouse v8.3.2 avec make.




Capture 1 — Versions des clients compilés, affichées dans Windows Terminal après interrogation SSH du VPS.

## 3. Configuration et services

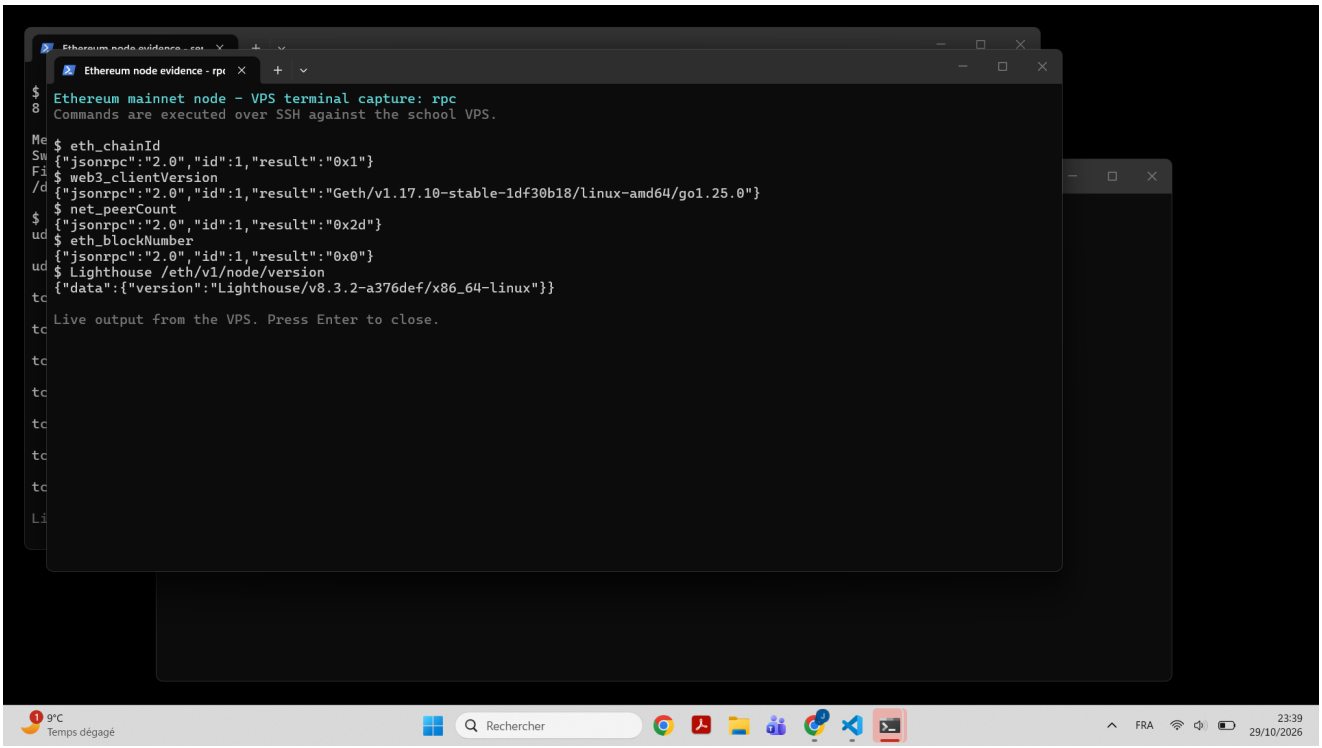
Geth est configuré sur mainnet en mode snap. Les RPC HTTP et WebSocket écoutent sur 8545 et 8546 avec les espaces de noms eth, net et web3. L'Engine API est limitée à localhost sur 8551 et utilise un secret JWT partagé avec Lighthouse. L'API Beacon écoute sur 5052 et les deux services systemd sont activés au démarrage.




Capture 2 — Ressources, stockage, services et ports constatés sur le VPS.

## 4. Vérification des endpoints

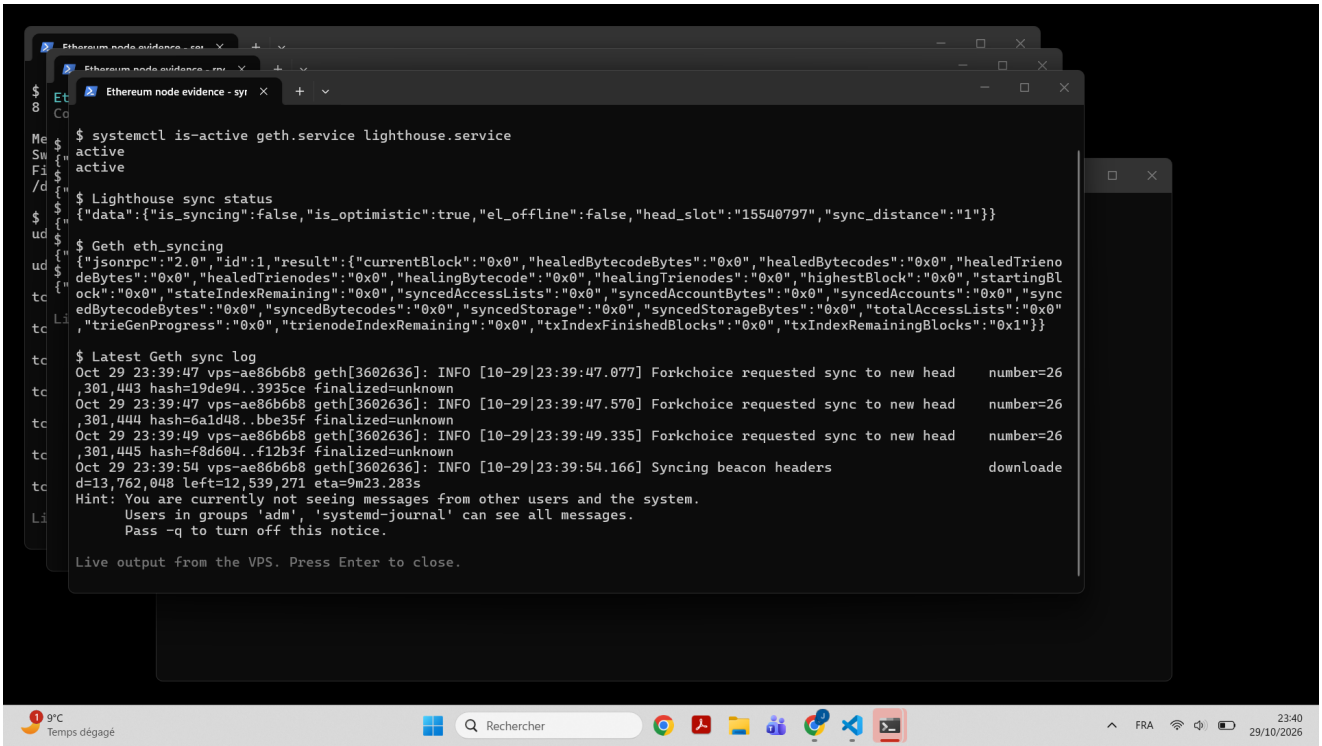
Les requêtes RPC exécutées sur le VPS confirment eth_chainId à 0x1 (mainnet), la version Geth, la présence de pairs et la version de l'API Beacon. Au moment de la capture, eth_blockNumber répond encore 0x0 : Geth n'a donc pas terminé la synchronisation d'exécution.




Capture 3 — Requêtes JSON-RPC et réponse de l'API Beacon depuis les clients actifs.

## 5. État de synchronisation

Au moment du relevé, les deux services étaient actifs. Lighthouse indiquait is_syncing=false, mais aussi is_optimistic=true et sync_distance=1. Geth renvoyait encore un bloc courant à 0x0 et ses journaux montraient le téléchargement des headers avec un retard important. La chaîne de consensus avançait, mais l'exécution n'était pas synchronisée ; je ne peux donc pas présenter le nœud comme complètement synchronisé.



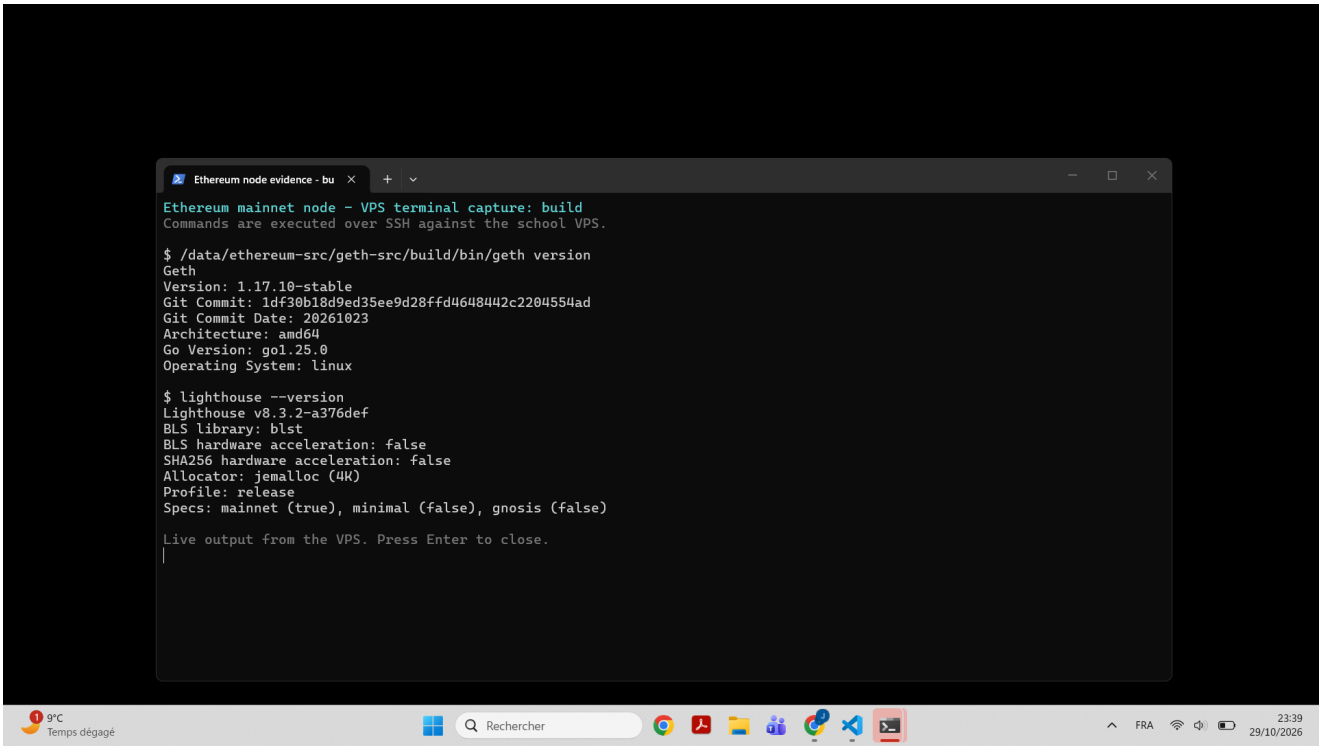

Capture 4 — État réel retourné par Lighthouse et Geth ; synchronisation incomplète au relevé.

## 6. Limite de stockage et conclusion

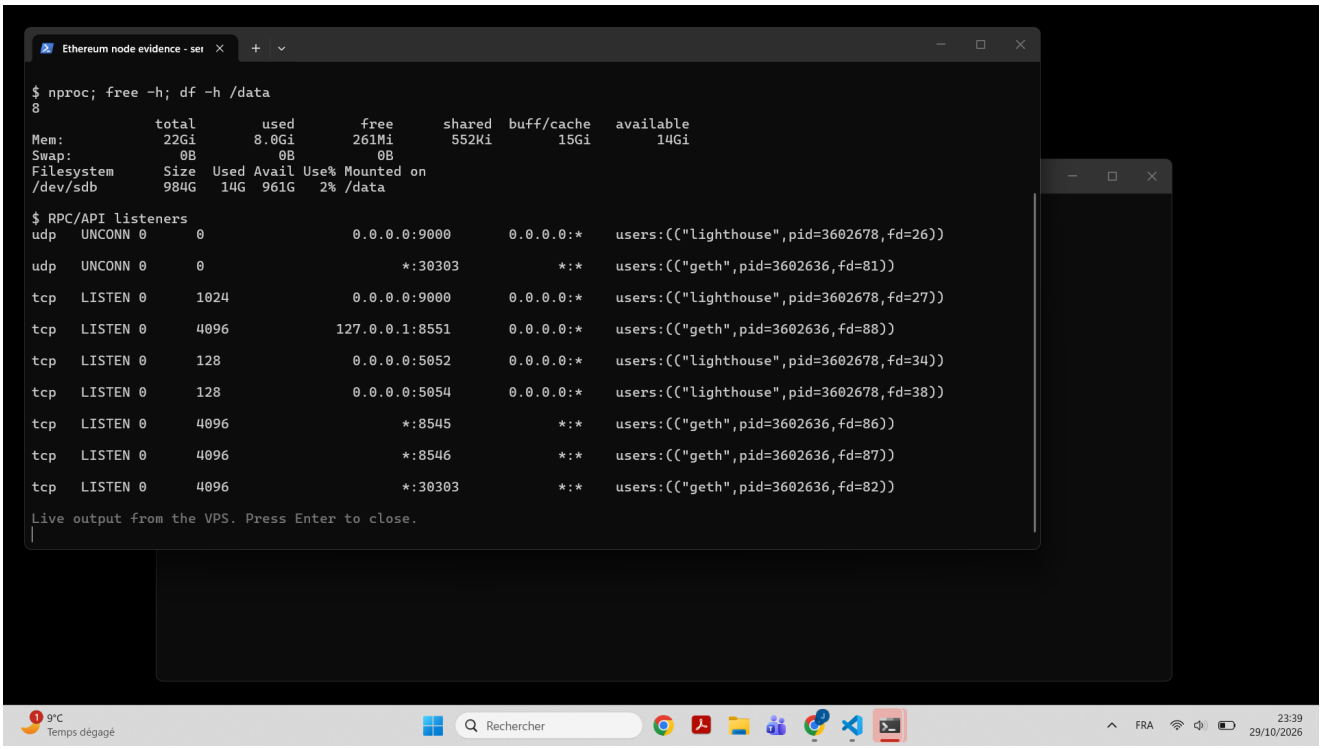
Le sujet demande 2 To de stockage disponible. Le volume /data de cette VM fait environ 984 Go, avec environ 961 Go libres au relevé. La contrainte matérielle n’est donc pas satisfaite et peut empêcher de conserver les données nécessaires à long terme. Les clients sont compilés et démarrés, les services systemd sont actifs et les endpoints répondent, mais la synchronisation complète n’est pas démontrée. Une ETA de journal est une estimation variable et ne permet pas de promettre une fin avant la remise du 31 octobre.

Les images de ce rapport sont des captures natives de l’écran Windows montrant Windows Terminal et la sortie de commandes exécutées par SSH sur la VM. Elles documentent l’état observé à l’heure de la capture et ne simulent pas une synchronisation achevée.

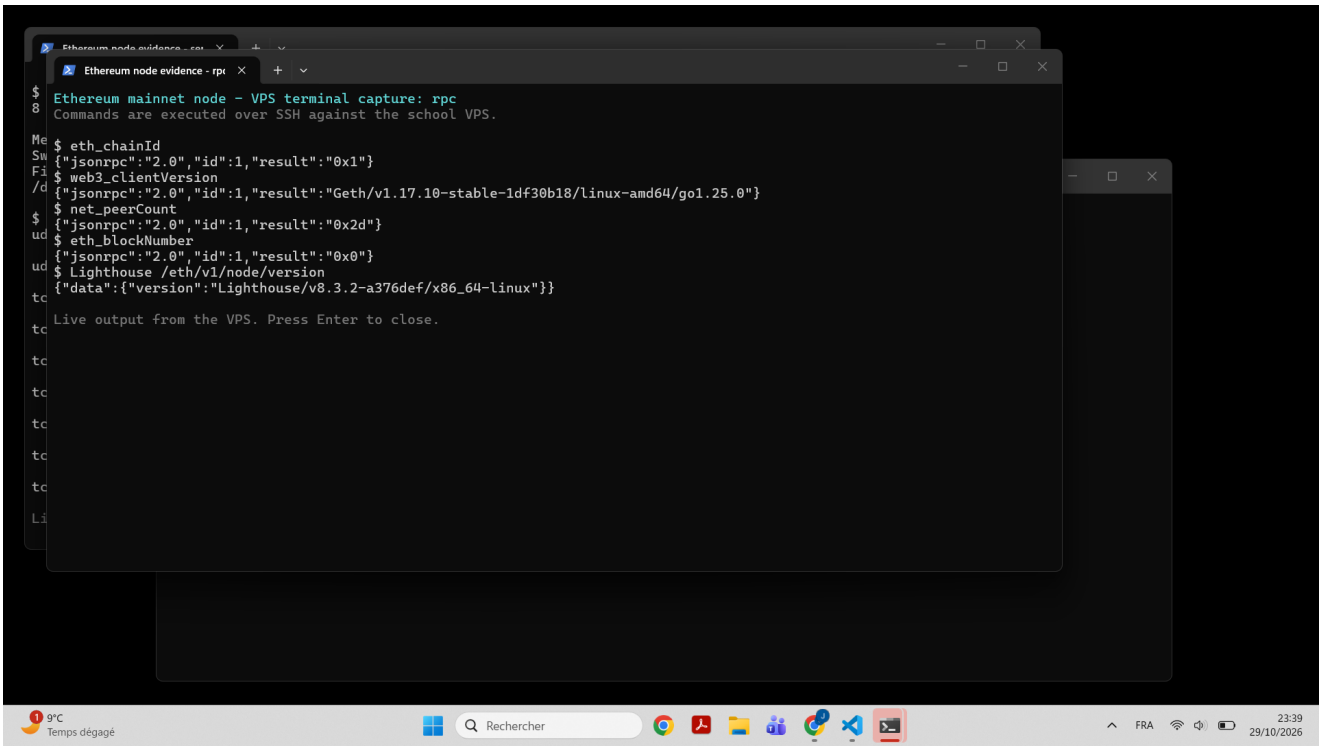
# Annexe — Captures du terminal



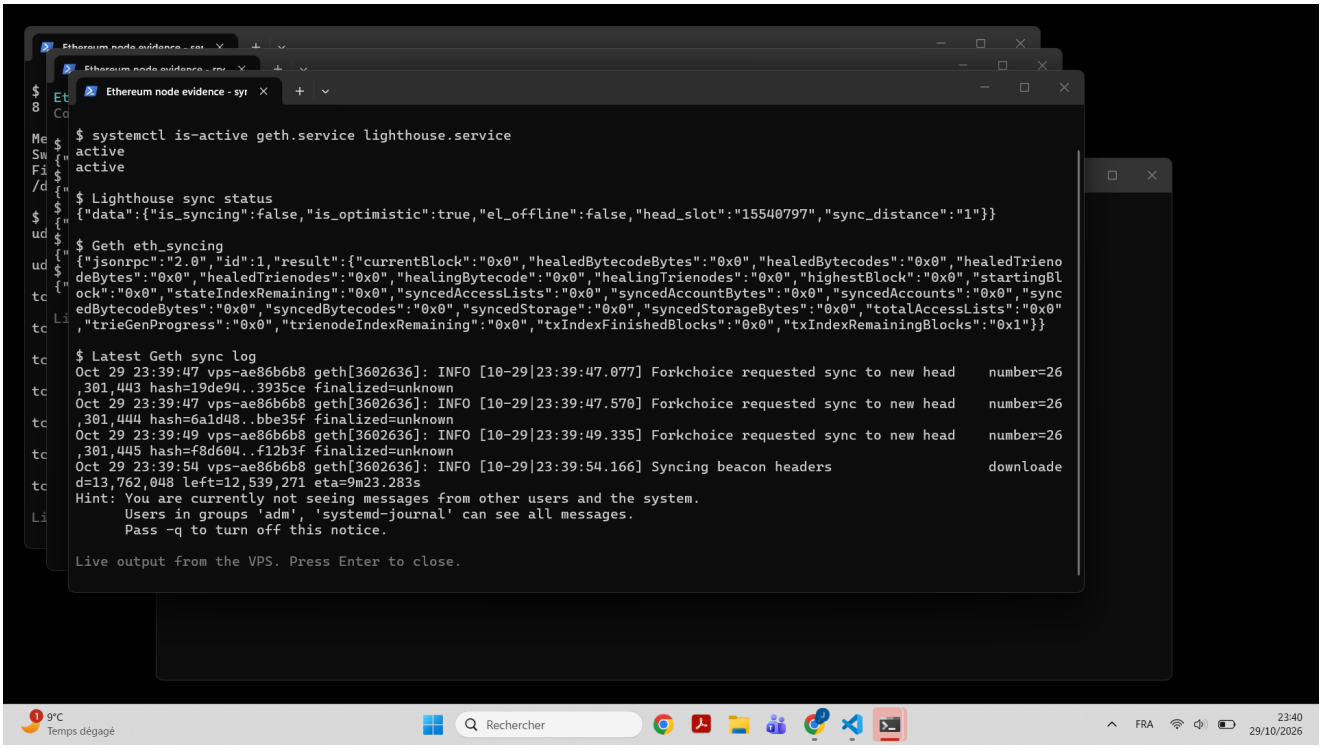

Clients compilés




Services et ressources




RPC et API Beacon




Synchronisation au relevé